

- JS5-函数
 - 5 函数记忆化
 - 定义
 - 适用场景
 - 核心原理
 - 基础实现
 - 使用JavaScript 对象
 - 使用Map
 - 支持 this 上下文
 - 限制缓存大小（LRU 缓存）

JS5-函数

5 函数记忆化

定义

函数记忆化（Memoization）是一种优化技术，它将函数的计算结果缓存起来，当后续使用相同的参数再次调用该函数时，直接返回缓存中的结果，避免重复计算。

适用场景

这种技术特别适合 **纯函数**（给定相同输入，总是返回相同输出，且无副作用）以及计算开销较大的场景。

核心原理

用空间换时间

通过一个缓存对象（如 JavaScript 对象或 Map）存储参数与返回值的映射关系。每次调用时：

1. 根据参数生成唯一的键（key）
2. 检查缓存中是否存在该键
3. 存在 → 返回缓存值

4. 不存在 → 执行原函数计算，将结果存入缓存，再返回结果

基础实现

使用JavaScript 对象

```
const fib = (n) => (n <= 1) ? 1 : (fib (n - 1) + fib (n - 2));

// 添加记忆化
function memoize(fn){
  const cache = {};
  return function(n){
    if(n in cache) return cache[n];
    cache[n] = fn(n);
    return cache[n];
  }
}

const memoizeFib = memoize(fib);
memoizeFib(40);

const date1 = Date.now();
console.log(memoizeFib(40));
const date2 = Date.now();
console.log(date2-date1);    // 7

const date3 = Date.now();
console.log(fib(40));
const date4 = Date.now();
console.log(date4-date3);    // 682
// 可以看到，记忆化函数能够避免重复计算，提高效率。
```

使用Map

```
const fib = (n) => (n <= 1) ? 1 : (fib (n - 1) + fib (n - 2));

// 添加记忆化
function memoize(fn){
  const cache = new Map();    // 用 Map 更灵活，键可以是任意类型
  return function(...args){
    const key = JSON.stringify(args);    // 简单序列化作为键
    if (cache.has(key)) {
      return cache.get(key);
    }
    const result = fn(args);
    cache.set(key, result);
    return result;
  }
}
```

```
const memoizeFib = memoize(fib);
memoizeFib(40);
```

`JSON.stringify` 有局限性：无法处理循环引用、函数、`undefined`、`Symbol` 等。可以自定义键生成函数，或者使用 `Map` 直接以参数作为键（参数数量固定时更简单）。

```
function memoizeSimple(fn) {
  const cache = new Map();
  return function(arg) { // 针对单参数
    if (cache.has(arg)) return cache.get(arg);
    const result = fn(arg);
    cache.set(arg, result);
    return result;
  };
}
```

对于多参数且参数是基本类型，可以使用 `args.join('|')` 或 `JSON.stringify`；也可使用 `WeakMap`。

支持 this 上下文

如果函数依赖 `this`，需要在包装时绑定：

```
function memoize(fn) {
  const cache = new Map();
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}
```

限制缓存大小（LRU 缓存）

无限增长的缓存可能导致内存泄漏。可以结合 LRU（Least Recently Used）策略，只保留最近使用的 `N` 条记录。简单实现示例：

```
class LRUMemoize {
  constructor(fn, maxSize = 100) {
    this.fn = fn;
    this.cache = new Map();
  }
}
```

```
    this.maxSize = maxSize;
  }

  get(...args) {
    const key = JSON.stringify(args);
    if (this.cache.has(key)) {
      // 刷新最近使用
      const value = this.cache.get(key);
      this.cache.delete(key);
      this.cache.set(key, value);
      return value;
    }
    const result = this.fn(...args);
    if (this.cache.size >= this.maxSize) {
      // 删除第一个（最旧的）键
      const firstKey = this.cache.keys().next().value;
      this.cache.delete(firstKey);
    }
    this.cache.set(key, result);
    return result;
  }
}
```